



Neural networks for total least squares solution of the time-varying linear systems

Xuezhong Wang¹ · Jiali Shan² · Yimin Wei³ 

Received: 3 November 2024 / Revised: 16 December 2024 / Accepted: 20 December 2024 /

Published online: 7 January 2025

© The Author(s) under exclusive licence to Sociedade Brasileira de Matemática Aplicada e Computacional 2025

Abstract

In this paper, we consider the total least squares solution of time-varying linear systems with time-varying right-hand side vectors. The neural network model termed as the neural network model for solving time-varying total least squares problems (NNTVTLS) and its discrete form of the neural network model for solving time-varying total least squares problems (DNNTVTLS) are provided to solve this problem. The analyses of convergence and robustness demonstrate that the proposed DNNTVTLS model exhibits global convergence and superior noise immunity. Numerical experiments further verify the superiority and effectiveness of the DNNTVTLS model in solving time-varying linear equations, taking into account the presence of noise.

Keywords Time-varying matrix · Time-varying linear system · Total least squares solution · Neural network model · Convergence · Robustness

Mathematics Subject Classification 15A18 · 15A69 · 65F15 · 65F10

Xuezhong Wang is supported by the National Natural Science Foundation of China under grant 12461055; The Natural Science Foundation of Gansu Province under grant 23JRRG0008. Jiali Shan is supported by the National Natural Science Foundation of China (Youth Student Basic Research Program) under grant 123B1008. Yimin Wei is supported by the Ministry of Science and Technology of China under grant H20240841 and the Joint Research Project between China and Serbia under grant 2024-6-7.

✉ Yimin Wei
ymwei@fudan.edu.cn; yimin.wei@gmail.com

Xuezhong Wang
Wangxuez@hxu.edu.cn; xuezhongwang77@126.com

Jiali Shan
21300180108@m.fudan.edu.cn

¹ School of Mathematics, Hexi University, Zhangye 734000, People's Republic of China

² School of Mathematical Sciences, Fudan University, Shanghai 200433, People's Republic of China

³ School of Mathematical Sciences and Key Laboratory of Mathematics for Nonlinear Sciences, Fudan University, Shanghai 200433, People's Republic of China

1 Introduction

Numerous challenges in science and technology sometimes require the solution of a set of linear equations

$$\bar{\mathbf{A}}\mathbf{x} = \bar{\mathbf{b}}, \quad (1.1)$$

where $\bar{\mathbf{A}} \in \mathbb{C}^{m \times n}$ is the data matrix and $\bar{\mathbf{b}} \in \mathbb{C}^n$ is the observation vector. In the least-squares (LS) approach, it is implicitly assumed that all errors are solely contained within the observation vector. However, this assumption frequently becomes impractical, as the data matrix itself is prone to various errors arising from sampling, human factors, modeling inaccuracies, and instrumental imperfections. To address both data and observation errors, the total least squares (TLS) method has been developed. The TLS method was initially introduced by Golub and Van Loan into the field of numerical analysis and they developed an algorithm based on the singular value decomposition (SVD) in Golub and Van Loan (2014), Golub and van Loan (1980), Luo and Unbehauen (1999). It has a rich history in statistical literature. Commonly referred to as orthogonal regression or errors-in-variables regression (EIV), this approach has been extensively researched over the past four decades, more detail in Cirrincione and Cirrincione (1999). A comprehensive analysis of the TLS problems is detailed in Huffel and Vandewalle (1991), where the algorithm introduced in Golub and van Loan (1980) is expanded to encompass all scenarios where it fails to yield a solution, specifically those pertaining to nongeneric TLS cases (Huffel and Vandewalle 1991). The TLS problem can be tackled using both direct and iterative methods. Direct methods involve the direct computation of the SVD. However, due to the computational complexity of the SVD, which amounts to $\mathcal{O}(m^2n + n^3)$, the practical application of TLS problems using direct methods is limited. Among the iterative methods, which are particularly suitable for slowly varying sets of equations, the most prominent techniques include the inverse iteration, ordinary and inverse Chebyshev iteration, Rayleigh Quotient iteration, and the Lanczos methods. For a detailed overview of these methods, the reader can refer to Huffel and Vandewalle (1991).

In the present study, our primary focus is on the following time-varying linear matrix equation,

$$\bar{\mathbf{A}}(t)\mathbf{x}(t) = \bar{\mathbf{b}}(t) \quad \text{with } t \in [0, +\infty). \quad (1.2)$$

Without loss of generality, $\bar{\mathbf{A}}(t)$ and $\bar{\mathbf{b}}(t)$ are assumed to be known, and their time derivatives $\dot{\bar{\mathbf{A}}}(t)$ and $\dot{\bar{\mathbf{b}}}(t)$ are assumed to be known or could be estimated. Time-varying linear equations emerge in various fields such as control theory, signal processing, model reduction, image restoration, ordinary and partial differential equations, as well as numerous applications in science and engineering (Kong and Feng 2024; Xiao 2015; Xiao and Zhang 2013). The references on the inverse or pseudoinverse of the time-varying linear systems can be found in Dai et al. (2023), Katsikis et al. (2022), Qi et al. (2025), Xiao et al. (2024).

The time-varying TLS problem for the linear equations assumes the perturbation of the coefficient matrix $\bar{\mathbf{A}}(t)$ by a time-varying matrix $\Delta\mathbf{A}(t)$ and the observation vector $\bar{\mathbf{b}}(t)$ by a time-varying vector $\Delta\mathbf{b}(t)$ for each t and tries to compensate for these, where $\Delta\mathbf{A}(t)$ and $\Delta\mathbf{b}(t)$ are noises, such as constant noises, time-varying linear noises, random noises, or their combination. The time-varying TLS method solves linear equations of the matrix form

$$\mathbf{A}(t)\mathbf{x}(t) \approx \mathbf{b}(t). \quad (1.3)$$

The time-varying TLS problem for the linear equations can be stated as the following optimization problem of finding the vector $\mathbf{x}_{TLS}^*$ that minimizes

$$\|(\Delta\mathbf{A})(t)\|_F^2 + \|(\Delta\mathbf{b})(t)\|^2,$$

with the constraint

$$[\bar{\mathbf{A}}(t) + (\Delta\mathbf{A})(t)]\mathbf{x}_{TLS}^* = \bar{\mathbf{b}}(t) + (\Delta\mathbf{b})(t).$$

The TLS problem can be equivalently formulated as the minimization (with respect to the vector $\mathbf{x}$) of the cost function (Cichocki and Unbehauen 1994; Lu et al. 2009)

$$\min_{\mathbf{x}(t) \in \mathbb{C}^n} f(\mathbf{x}(t)) := \frac{\|\mathbf{r}(\mathbf{x}(t))\|^2}{1 + \mathbf{x}(t)^\top \mathbf{x}(t)}, \quad (1.4)$$

where $\mathbf{r}(\mathbf{x}(t)) = \mathbf{A}(t)\mathbf{x}(t) - \mathbf{b}(t)$ with $\mathbf{A}(t) = \bar{\mathbf{A}}(t) + (\Delta\mathbf{A})(t)$, $\mathbf{b}(t) = \bar{\mathbf{b}}(t) + (\Delta\mathbf{b})(t)$ and $\|\mathbf{r}(\mathbf{x}(t))\|$ is the spectral norm of $\mathbf{r}(\mathbf{x}(t))$.

If the perturbation matrix satisfies $\Delta\mathbf{A}(t) = \mathbf{O}$, where $\mathbf{O}$ denotes the zero matrix, then the TLS problem reduces to the least squares (LS) for the time-varying linear equations. In comparison with the standard LS approach, to obtain the solution of the TLS problem is generally quite burdensome and time consuming (Fierro et al. 1997; Golub et al. 1999; Golub and van Loan 1980; Han et al. 2024; Huffel and Vandewalle 1991; Liu et al. 2022; Lu et al. 2009; Xie et al. 2017, 2018; Zheng et al. 2017). Therefore, our first motivation is to propose a simple and efficient algorithm to solve the TLS problem by using only one single artificial neural network.

The time-varying problems have drawn much attention for more than ten years, because it can be applied to solve the inverse-kinematic problem arising from the robot manipulator operating (Guo et al. 2011; Li and Li 2014; Shen et al. 2015; Xiao 2015; Xiao and Zhang 2013; Zhang et al. 2002; Zhang and Yi 2011), in which the Zhang neural networks, ZNNs for short, are used. Compared with the traditional numerical methods and gradient-based optimization methods called gradient-based neural network (GNN), the ZNN methods have many advantages.

Generally, in the GNN method, ℓ_2 norm of the error function is used as a criterion to measure the performance of the decreasing errors. Since this method is based on the negative gradient-descent direction only and does not use the velocity information sufficiently, as a result, the norm of the ℓ_2 error function does not decline to zero for time-varying problems even if the time span is sufficiently long (Xiao and Zhang 2013). However, as time evolves, the errors computed by the ZNN method diminish to zero quickly which shows a substantial superiority over the GNN approaches (Guo et al. 2011; Wang et al. 2016; Xiao and Zhang 2011, 2012; Zhang and Ge 2005; Zhang et al. 2002, 2009), thus it is efficient for solving the time-varying problems. Due to these reasons, the ZNN method is popular in studying the time-varying problems and it has been applied to many problems, such as Sylvester and Lyapunov equations (Shen et al. 2015), matrix inversions and generalized inverses (Qiao et al. 2018, 2020; Wang et al. 2016; Zhang and Ge 2005; Zhang et al. 2002), matrix and tensor eigenpairs (Mo et al. 2020), tensor equations (Wang et al. 2022) and so on.

Furthermore, to expand the capabilities of the ZNN model, Jin et al. (2016) and Liao et al. (2019) have introduced integration-enhanced continuous time ZNN (CTZNN) models, specifically tailored for tackling diverse dynamic challenges amidst noisy environments. These advanced ZNN approaches have demonstrated remarkable success across various domains. Lei et al. (2022) further contributed by developing a CTZNN model equipped with anti-noise features, effectively addressing complex matrix generalized inverses under varying noise conditions. Liao et al. (2021), leveraging two nonlinear activation functions, introduced a robust class of nonlinear ZNN models, successfully applied to solve dynamic problems in noisy settings. To combat intricate linear noise, a complex-valued noise-tolerant zeroing neural network on the basis of an integral-type design formula is established and investigated for finding complex-valued time-dependent matrix inversion under a wide variety of noises

in Xiao et al. (2020). All these modified models ensure convergence within finite time and effectively eliminate external noise.

Despite the extensive research on neurodynamics and its successful application to continuous models for dynamic problems, directly implementing continuous-time neurodynamics on digital computers, which operate with discrete time steps, poses challenges. Conversely, discrete-time neurodynamics has emerged as a more suitable choice for computer implementations, particularly for online applications, garnering significant attention in recent years (Jin and Zhang 2014; Perez-Ilzarbe 1998; Petković et al. 2018; Qiu and Zhang 2019; Shi et al. 2023, 2022; Zhang et al. 2021). For example, a discrete-time neurodynamics approach was introduced in Perez-Ilzarbe (1998) for quadratic optimization with bound constraints. By integrating a scaled Hyperpower iterative algorithm, a discrete-time model was proposed for solving matrix generalized inverse (Petković et al. 2018). Shi et al. (2023) developed a discrete-time recurrent neurodynamics model based on the Taylor expansion for dynamic matrix inversion. Additionally, Qiu and Zhang (2019) utilized a six-instant discretization formula in two discrete-time neurodynamic models for online computation of future matrix inversions.

It is crucial to observe that prior discrete-time algorithmic investigations primarily concentrated on tackling dynamic challenges in noise-free settings, neglecting the repercussions of noise in contemporary algorithms. Nonetheless, practical implementations invariably encounter interference noise, which undermines system convergence and robustness, attributes unattainable by the earlier frameworks. Designing a precise discrete-time neurodynamics approach equipped with robust denoising capabilities stands as a formidable yet worthwhile endeavor. This advanced methodology holds the key to addressing intricate real-world problems, particularly in contexts where noise dominates. By prioritizing precision and denoising abilities, we enhance the model's reliability and robustness, bolstering its real-world applicability. One path to realizing this methodology involves discretizing continuous-time denoising neurodynamics, enabling it to grapple with online, dynamic amidst diverse, intricate noise landscapes. This strategy acknowledges the complexity of real-world environments where noise is an intrinsic component of system dynamics. Therefore, our second motivation is to design a precise discrete-time neurodynamics approach equipped with robust denoising capabilities to solve the TLS problems of time-varying linear equations.

Overall, the primary motivation in this research is to design neural network models directly for computing the total least squares solution of the time-varying linear equations (TVLE) (1.2). For convenience of the reader, we briefly summarize our main contributions and novelties as follows,

1. We focus on the total least square of the time-varying linear equations (1.2) where the data matrix flows $\mathbf{A}(t)$ and observation vector flows $\mathbf{b}(t)$ are time-varying and containing errors, while the existing results are mainly total least squares solution with the data matrix $\mathbf{A}$ and observation vector $\mathbf{b}$.
2. We prove the presented discrete-time neurodynamic models are convergent to the total least squares solution of the time-varying linear equations with small errors.
3. The theoretical analysis underscores the robust denoising capabilities of the DNNTVTLS model (3.8) when confronted with diverse noise types. Additionally, it demonstrates superior accuracy in intricate, noisy environments.
4. The presented numerical experimental results are analyzed and discussed, reinforcing the advantageous traits of the proposed DNNTVTLS model (3.8) across a wide range of the noisy settings.

This paper is organized into seven sections. In Sect. 2, some notations and basic results are recalled for dealing with the tensor problem. In Sect. 3, we present detailed procedures on formulating our models. Shortly afterwards convergence analysis of the neural network models is given in Sect. 4. In Sect. 5, several representative examples are given to illustrate the efficiency of those neural network models, and we conclude our paper in the last section.

2 Preliminaries

We give a short description of the notations and recall some results that are frequently used.

Throughout this paper, we use capital letters to denote matrix, and vectors are denoted by bold lower letters. $\|\mathbf{A}\|_F$, $\mathbf{A}^\top$, $\mathbf{A}^H$, $\Re(\mathbf{A})$ and $\Im(\mathbf{A})$ denote the Frobenius norm, the transpose, the complex conjugated transpose, the real part and the imaginary part of a given matrix $\mathbf{A} \in \mathbb{C}^{m \times n}$, respectively. This notation is consistently used for lower-order parts of a given structure. For example, the entry with row index i and column index j in a matrix $\mathbf{A}$, i.e., $\mathbf{A}_{ij}$, is symbolized by a_{ij} (also $(\mathbf{x})_i = x_i$). Hence, we use $|\mathbf{A}| = (|a_{kj}|)$, $\Theta(\mathbf{A}) = (\Theta(a_{kj}))$ and $\exp(\mathbf{A}) = (\exp(a_{kj}))$ denote the element-wise modulus, the element-wise argument and the element-wise exponential of the matrix $\mathbf{A} \in \mathbb{C}^{m \times n}$, respectively. We use $\mathbf{I}_n$ to denote the $n \times n$ identical matrix.

The singular-value decomposition (SVD) of the matrix $[\mathbf{A}(t), \mathbf{b}(t)]$ is denoted by Bunse-Gerstner et al. (1991)

$$[\mathbf{A}(t), \mathbf{b}(t)] = \mathbf{U}(t)\Sigma(t)\mathbf{V}(t)^\top, \quad (2.1)$$

where

$$\begin{cases} \mathbf{U}(t) = [\mathbf{u}_1(t), \mathbf{u}_2(t), \dots, \mathbf{u}_m(t)], & \mathbf{u}_i(t) \in \mathbb{R}^m, & \mathbf{U}(t)^\top \mathbf{U}(t) = \mathbf{U}(t)\mathbf{U}(t)^\top = \mathbf{I}_m, \\ \mathbf{V}(t) = [\mathbf{v}_1(t), \mathbf{v}_2(t), \dots, \mathbf{v}_{n+1}(t)], & \mathbf{v}_i(t) \in \mathbb{R}^{n+1}, & \mathbf{V}(t)^\top \mathbf{V}(t) = \mathbf{V}(t)\mathbf{V}(t)^\top = \mathbf{I}_{n+1}, \\ \Sigma(t) = \text{diag}(\delta_1(t), \delta_2(t), \dots, \delta_{n+1}(t)) \in \mathbb{R}^{m \times (n+1)} & \text{with } \delta_1(t) \geq \delta_2(t) \geq \dots \geq \delta_{n+1}(t) \geq 0. \end{cases}$$

The TLS problem is defined as basic if verify (a) it is unidimensional, (b) it is solvable and (c) it has a unique solution.

Given the overdetermined systems (1.3), the TLS solution is given by Fierro et al. (1997), Golub and Van Loan (2014)

$$\mathbf{x}_{TLS}(t) = -\frac{1}{v_{n+1,n+1}}(v_{1,n+1}, v_{2,n+1}, \dots, v_{n,n+1})^\top, \quad (2.2)$$

where $\mathbf{v}_{n+1}$ is the right singular vector corresponding to the smallest singular value of $[\mathbf{A}(t), \mathbf{b}(t)]$ and $v_{n+1,n+1}$ is the last component of $\mathbf{v}_{n+1}$.

The TLS solution is parallel to the right singular vector corresponding to the minimum singular value of $[\mathbf{A}(t); \mathbf{b}(t)]$, which can be expressed as a Rayleigh quotient with respect to the spectral norm,

$$\delta_{n+1}(t)^2 = \frac{\|\mathbf{Ax}(t) - \mathbf{b}(t)\|^2}{1 + \mathbf{x}(t)^\top \mathbf{x}(t)}. \quad (2.3)$$

3 Neural networks for solving the TLS

With the help of specific activation functions, we present the neural network model for solving TVTLS (1.4) in this section.

It is assumed that each entry of the TV matrix flows $\mathbf{A}(t) = \bar{\mathbf{A}}(t) - (\Delta\mathbf{A})(t)$ and the vector flows $\mathbf{b}(t) = \bar{\mathbf{b}}(t) - (\Delta\mathbf{b})(t)$ are differentiable with respect to the parameter t .

3.1 Neural network models

We adopt the main idea of the gradient neural network (GNN) models and Zhang neural network (ZNN) model introduced in Wang et al. (2022) for generating our neural network models (NN) to solve the TVTLS (1.4).

For convenience, let $\mathbf{r}(\mathbf{x}(t)) = \mathbf{A}(t)\mathbf{x}(t) - \mathbf{b}(t)$. Note that our aim is to solve the minimum optimization problem (Cichocki and Unbehauen 1994)

$$\min_{\mathbf{x}(t) \in \mathbb{C}^n} \delta(\mathbf{x}(t)) = \frac{\|\mathbf{r}(\mathbf{x}(t))\|^2}{1 + \mathbf{x}(t)^\top \mathbf{x}(t)}.$$

Unlike gradient-based neural network models, which find the minimum of a function $\delta(\mathbf{x}(t))$ by following the direction of the negative gradient, this approach ignores temporal information. We establish an error function that incorporates temporal factors. Let

$$\mathbf{e}(\mathbf{x}(t)) = \frac{\mathbf{r}(\mathbf{x}(t))}{\sqrt{1 + \mathbf{x}(t)^\top \mathbf{x}(t)}}, \quad (3.1)$$

in which $\mathbf{e}(\mathbf{x}(t)) \in \mathbb{R}^m$, the function $\mathbf{e}(\mathbf{x}(t))$ acts as a dual monitoring and control mechanism. By precisely managing and reducing $\mathbf{e}(\mathbf{x}(t))$ towards zero, the resulting value of $\mathbf{x}(t)$ steadily converges towards the theoretical solution. Therefore, we use the formula (Wang et al. 2022)

$$\frac{d(\mathbf{e}(\mathbf{x}(t)))}{dt} = -\gamma \Psi(t) \mathbf{e}(\mathbf{x}(t)), \quad (3.2)$$

where γ is positive scaling constant and $\Psi(t) = \exp(\alpha_1 t + \frac{\alpha_2}{1+t} + \alpha_3 \|\mathbf{r}(\mathbf{x}(t))\|)$ with $\alpha_1, \alpha_2, \alpha_3 \geq 0$ being the time-varying parameters. The terms $\alpha_1 t$ and $\frac{\alpha_2}{1+t}$ in $\Psi(t)$ are designed to relieve the model-implementation noise, especially for some unbounded noises, which will be verified in the numerical experiments. The term $\alpha_3 \|\mathbf{e}(\mathbf{x}(t))\|$ in the varying parameter $\Psi(t)$ is designed to reduce the effect of the initial state. Computing the derivative of $\mathbf{e}(\mathbf{x}(t))$ at time t , we have

$$\frac{d\mathbf{e}(\mathbf{x}(t))}{dt} = \frac{\partial \mathbf{e}(\mathbf{x}(t))}{\partial t} + \frac{\partial \mathbf{e}(\mathbf{x}(t))}{\partial \mathbf{x}(t)} \frac{d\mathbf{x}(t)}{dt}. \quad (3.3)$$

Since $\mathbf{e}(\mathbf{x}(t))$ is continuous and differentiable with respect to $\mathbf{x}(t)$, the following formula is obtained

$$\frac{\partial \mathbf{e}(\mathbf{x}(t))}{\partial \mathbf{x}(t)} = \frac{\mathbf{A}(t)}{\sqrt{1 + \mathbf{x}(t)^\top \mathbf{x}(t)}} - \frac{\mathbf{r}(\mathbf{x}(t))\mathbf{x}(t)^\top}{(1 + \mathbf{x}(t)^\top \mathbf{x}(t))^{\frac{3}{2}}}. \quad (3.4)$$

Combining (3.2) with (3.3) and (3.4), we obtain the following neural network model,

$$\left(\frac{\mathbf{A}(t)}{\sqrt{1 + \mathbf{x}(t)^\top \mathbf{x}(t)}} - \frac{\mathbf{r}(\mathbf{x}(t))\mathbf{x}(t)^\top}{(1 + \mathbf{x}(t)^\top \mathbf{x}(t))^{\frac{3}{2}}} \right) \frac{d\mathbf{x}(t)}{dt} = -\gamma \Psi(t) \frac{\mathbf{r}(\mathbf{x}(t))}{\sqrt{1 + \mathbf{x}(t)^\top \mathbf{x}(t)}} - \frac{\dot{\mathbf{A}}(t)\mathbf{x}(t) - \dot{\mathbf{b}}(t)}{\sqrt{1 + \mathbf{x}(t)^\top \mathbf{x}(t)}}. \quad (3.5)$$

By multiplying both sides of (3.5) by $\sqrt{1 + \mathbf{x}(t)^\top \mathbf{x}(t)}$, we obtain

$$\left(\mathbf{A}(t) - \frac{\mathbf{r}(\mathbf{x}(t))\mathbf{x}(t)^\top}{1 + \mathbf{x}(t)^\top \mathbf{x}(t)} \right) \frac{d\mathbf{x}(t)}{dt} = -\gamma \Psi(t) \mathbf{r}(\mathbf{x}(t)) - (\dot{\mathbf{A}}(t)\mathbf{x}(t) - \dot{\mathbf{b}}(t)). \quad (3.6)$$

Furthermore, we have

$$\frac{d\mathbf{x}(t)}{dt} = \left(\mathbf{A}(t) - \frac{\mathbf{r}(\mathbf{x}(t))\mathbf{x}(t)^\top}{1 + \mathbf{x}(t)^\top \mathbf{x}(t)} \right)^\dagger (-\gamma \Psi(t) \mathbf{r}(\mathbf{x}(t)) - (\dot{\mathbf{A}}(t)\mathbf{x}(t) - \dot{\mathbf{b}}(t))), \quad (3.7)$$

where $\mathbf{B}^\dagger$ denotes the Moore–Penrose inverse (Wei et al. 2018) of the matrix $\mathbf{B}$. The dynamic equation (3.7) will be termed as neural network model for solving time-varying total least squares problems, simplified NNTVTLS.

3.2 Discretize form of NNTVTLS

The Euler difference formula serves as an efficient technique to discretize the NNTVTLS model. Consequently, we substitute the Euler forward difference approximation for $\dot{\mathbf{x}}(t)$ in equation (3.7), along with the Euler backward difference approximation for $\dot{\mathbf{A}}(t)$ and $\dot{\mathbf{b}}(t)$. Let $\theta = \gamma\tau$, where τ denotes the step. Thus we have the discretize form of the NNTVTLS model

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \left(\mathbf{A}_k - \frac{\mathbf{r}_k \mathbf{x}_k^\top}{1 + \mathbf{x}_k^\top \mathbf{x}_k} \right)^\dagger (\theta \Psi(k\tau) \mathbf{r}_k - \tau ((\mathbf{A}_k - \mathbf{A}_{k-1}) \mathbf{x}_k - (\mathbf{b}_k - \mathbf{b}_{k-1}))). \quad (3.8)$$

The dynamic equation (3.8) will be termed the discretize neural network model for solving time-varying total least squares problems, simplified DNNTVTLS. It is noteworthy that varying parameter values, including τ , lead to discernible differences in the convergence performance of the DNNTVTLS model (3.8). This distinction becomes apparent in the subsequent numerical examples.

4 Theoretical results

In this section, we delve into the analysis of the convergence and robustness of the proposed DNNTVTLS model (3.8) under varying noise conditions. As a precursor to the subsequent theoretical proofs, the DNNTVTLS model (3.8) is first simplified to establish a solid foundation for the analysis.

4.1 Convergence of the DNNTVTLS model

Theorem 4.1 *The DNNTVTLS model (3.8) is simplified as follows,*

$$(1 + \theta \Psi(k\tau)) \mathbf{e}_k - \mathbf{e}_{k-1} + \mathbf{O}(\tau^2) = \mathbf{0}, \quad (4.1)$$

where $\mathbf{e}_k = \frac{\mathbf{A}_k \mathbf{x}_k - \mathbf{b}_k}{\sqrt{1 + \mathbf{x}_k^\top \mathbf{x}_k}}$ and $\mathbf{O}(\tau^2) \in \mathbb{R}^m$ with each of its element being $\mathcal{O}(\tau^2)$, $\mathbf{O}(\tau^2)$ signifies that the truncation error is of the order of τ^2 .

Proof From (3.8), we obtain

$$\left(\mathbf{A}_k - \frac{\mathbf{r}_k \mathbf{x}_k^\top}{1 + \mathbf{x}_k^\top \mathbf{x}_k} \right) (\mathbf{x}_{k+1} - \mathbf{x}_k) = -(\theta \Psi(k\tau) \mathbf{r}_k - \tau ((\mathbf{A}_k - \mathbf{A}_{k-1}) \mathbf{x}_k - (\mathbf{b}_k - \mathbf{b}_{k-1}))). \quad (4.2)$$

It follows from the Taylor expansion that

$$\mathbf{x}_{k+1} - \mathbf{x}_k = \tau \dot{\mathbf{x}}_k + \mathbf{O}(\tau^2). \quad (4.3)$$

Together with (4.2) and (4.3), we obtain the following equation

$$\left(\mathbf{A}_k - \frac{\mathbf{r}_k \mathbf{x}_k^\top}{1 + \mathbf{x}_k^\top \mathbf{x}_k} \right) (\tau \dot{\mathbf{x}}_k + \mathbf{O}(\tau^2)) = -(\theta \Psi(k\tau) \mathbf{r}_k - \tau ((\mathbf{A}_k - \mathbf{A}_{k-1}) \mathbf{x}_k - (\mathbf{b}_k - \mathbf{b}_{k-1}))). \quad (4.4)$$

Furthermore, we get

$$\left(\mathbf{A}_k - \frac{\mathbf{r}_k \mathbf{x}_k^\top}{1 + \mathbf{x}_k^\top \mathbf{x}_k} \right) (\tau \dot{\mathbf{x}}_k + \mathbf{O}(\tau^2)) = -(\theta \Psi(k\tau) \mathbf{r}_k - \tau (\dot{\mathbf{A}}_k \mathbf{x}_k - \dot{\mathbf{b}}_k)). \quad (4.5)$$

Thus

$$\left(\mathbf{A}_k - \frac{\mathbf{r}_k \mathbf{x}_k^\top}{1 + \mathbf{x}_k^\top \mathbf{x}_k} \right) \dot{\mathbf{x}}_k + \dot{\mathbf{A}}_k \mathbf{x}_k - \dot{\mathbf{b}}_k = -\frac{\theta \Psi(k\tau)}{\tau} \mathbf{r}_k + \mathbf{O}(\tau^2). \quad (4.6)$$

It follows from formula (3.5) that

$$\dot{\mathbf{e}}_k = -\frac{\theta \Psi(k\tau)}{\tau} \mathbf{e}_k + \mathbf{O}(\tau^2). \quad (4.7)$$

By employing the Euler forward difference formula, we obtain

$$\frac{\mathbf{e}_k - \mathbf{e}_{k-1}}{\tau} = -\frac{\theta \Psi(k\tau)}{\tau} \mathbf{e}_k + \mathbf{O}(\tau^2). \quad (4.8)$$

Then

$$(1 + \theta \Psi(k\tau)) \mathbf{e}_k - \mathbf{e}_{k-1} + \mathbf{O}(\tau^2) = \mathbf{0}. \quad (4.9)$$

□

Next, we analyze the convergence of the DNNTVTLS model (3.8).

Theorem 4.2 *The DNNTVTLS model (3.8) exhibits a steady-state error $\lim_{k \rightarrow \infty} \|\mathbf{e}_k\|$ which is $\mathcal{O}(\tau^2)$ when employed for solving the time-varying linear equation (1.3) without noise.*

Proof Express the m -th subsystem of $\mathbf{e}_k$ in terms of $\mathbf{e}_k^m$, where m indicates the m -th component of $\mathbf{e}_k$. Additionally, the m -th subsystem of (4.9) is determined at the time instance $k\tau$.

$$(1 + \theta \Psi(k\tau)) \mathbf{e}_k^m - \mathbf{e}_{k-1}^m + \mathbf{O}(\tau^2) = 0. \quad (4.10)$$

The equation (4.10) is transformed into

$$\mathbf{e}_k^m = \frac{1}{(1 + \theta \Psi(k\tau))} \mathbf{e}_{k-1}^m + \mathbf{O}(\tau^2). \quad (4.11)$$

Thus, one obtain the following relation

$$\begin{aligned} \mathbf{e}_k^m &= \frac{1}{(1 + \theta \Psi(k\tau))} \mathbf{e}_{k-1}^m + \mathcal{O}(\tau^2) \\ &= \frac{1}{(1 + \theta \Psi(k\tau))(1 + \theta \Psi((k-1)\tau))} \mathbf{e}_{k-2}^m + \mathcal{O}(\tau^2) \\ &= \frac{1}{(1 + \theta \Psi(k\tau))(1 + \theta \Psi((k-1)\tau)) \dots (1 + \theta \Psi(\tau))} \mathbf{e}_0^m + \mathcal{O}(\tau^2). \end{aligned} \quad (4.12)$$

Let $\theta_{\max} = \max\{1 + \theta \Psi(k\tau), 1 + \theta \Psi((k-1)\tau), \dots, 1 + \theta \Psi(\tau)\}$. Note that $\theta > 0$ and $\Psi(j\tau) \geq 1$, $j = 1, 2, \dots, k$ and we get $\theta_{\max} > 1$. Therefore,

$$\begin{aligned} \mathbf{e}_k^m &= \frac{1}{(1 + \theta \Psi(k\tau))(1 + \theta \Psi((k-1)\tau)) \dots (1 + \theta \Psi(\tau))} \mathbf{e}_0^m + \mathcal{O}(\tau^2) \\ &\leq \left(\frac{1}{\theta_{\max}}\right)^k \mathbf{e}_0^m + \mathcal{O}(\tau^2). \end{aligned}$$

Thus, $\lim_{k \rightarrow \infty} (\frac{1}{\theta_{\max}})^k = 0$, follows from $\frac{1}{\theta_{\max}} < 1$. Then, we further obtain

$$\lim_{k \rightarrow \infty} \|\mathbf{e}_k^m\| = \mathcal{O}(\tau^2).$$

Then it is straightforward to deduce that the steady-state error of the noise-free DNNTVTLs model (3.8), as k approaches infinity $\lim_{k \rightarrow \infty} \|\mathbf{e}_k\|$, is of the order $\mathcal{O}(\tau^2)$. $\square$

4.2 Robustness analysis

The DNNTVTLs model (3.8) undergoes rigorous theoretical robustness analyses under various noise conditions, demonstrating its significant resilience and immunity to noise.

Theorem 4.3 *Under the pollution of linear noise $\mathbf{n}_k = \mathbf{p}k\tau + \mathbf{u}$, the steady-state error $\lim_{k \rightarrow \infty} \|\mathbf{e}_k\|$ of the DNNTVTLs model (3.8) for solving the time-varying TLS (1.3) is $\|\mathbf{p}\tau\| + \mathcal{O}(\tau^2)$, where $\mathbf{p}, \mathbf{u} \in \mathbb{R}^n$.*

Proof According to the principle of superposition, the steady-state error $\|\mathbf{p}\tau\| + \mathcal{O}(\tau^2)$ of the DNNTVTLs model (3.8), which is contaminated by the linear noise when solving the time-varying TLS (1.3), is segregated into two components: one being $\mathcal{O}(\tau^2)$ stemming from the residual error, and the other originating from the linear noise. Based on Theorem 4.1, we understand that $\mathcal{O}(\tau^2)$ is attributed to the $\mathcal{O}(\tau^2)$ term. Consequently, our focus should solely be on the portion generated by the linear noise, which can be expressed by an equation analogous to (4.1),

$$(1 + \theta\Psi(k\tau))\mathbf{e}_k - \mathbf{e}_{k-1} + \mathbf{p}k\tau + \mathbf{u} = \mathbf{0}. \quad (4.13)$$

The equation mentioned below serves as a further generalization of the aforementioned statement.

$$(1 + \theta\Psi((k+1)\tau))\mathbf{e}_{k+1} - \mathbf{e}_k + \mathbf{p}(k+1)\tau + \mathbf{u} = \mathbf{0}. \quad (4.14)$$

The m -th subsystem of (4.14) can be expressed as,

$$(1 + \theta\Psi((k+1)\tau))\mathbf{e}_{k+1}^m - \mathbf{e}_k^m + \mathbf{p}^m(k+1)\tau + \mathbf{u}^m = 0. \quad (4.15)$$

The Z-transform of the above equation has the form

$$(1 + \theta\Psi((k+1)\tau))(z\mathbf{e}^m(z) - z\mathbf{e}^m(0)) - \mathbf{e}^m(z) + \frac{z\mathbf{p}^m\tau}{(z-1)^2} + \frac{z\mathbf{u}^m}{z-1}. \quad (4.16)$$

After rearrangement, (4.16) leads to

$$\mathbf{e}^m(z) = \frac{(z-1)^2(1 + \theta\Psi((k+1)\tau))\mathbf{e}^m(0) - z\mathbf{p}^m\tau - z(z-1)\mathbf{u}^m}{(z-1)((z-1)(\theta\Psi((k+1)\tau) + z))}. \quad (4.17)$$

The above function has the poles, formulated as $z_1 = 1$ and $z_2 = \frac{\theta\Psi((k+1)\tau)}{1 + \theta\Psi((k+1)\tau)}$. Note that both poles lie within or on the unit circle, thereby yielding

$$\begin{aligned} \lim_{k \rightarrow \infty} \mathbf{e}_k^m &= \lim_{z \rightarrow 1} (z-1)\mathbf{e}^m(z) \\ &= \lim_{z \rightarrow 1} \frac{(z-1)^2(1 + \theta\Psi((k+1)\tau))\mathbf{e}^m(0) - z\mathbf{p}^m\tau - z(z-1)\mathbf{u}^m}{((z-1)(\theta\Psi((k+1)\tau) + z))} \\ &= -\mathbf{p}^m\tau. \end{aligned}$$

In summary, we have shown that the steady-state error of the DNNTVTLs model (3.8) in solving the time-varying TLS (1.3), which is corrupted by linear noise $\mathbf{p}k\tau + \mathbf{u}$, is $\|\mathbf{p}\tau\| + \mathcal{O}(\tau^2)$. $\square$

Remark 4.4 If the coefficient $\mathbf{p}$ of linear noise $\mathbf{p}k\tau + \mathbf{u}$ is $\mathbf{0}$, we can obtain the steady-state error of the DNNTVTL model (3.8) for solving the time-varying TLS (1.3) polluted by constant noise $\mathbf{n}_k = \mathbf{u}$, which is evidently $\mathcal{O}(\tau^2)$.

Furthermore, when it is contaminated by bounded random noise $\mathbf{n}_k = \sigma \in \mathbb{R}^n$, the following theorem holds.

Theorem 4.5 Under the pollution of bounded random noise $\mathbf{n}_k = \sigma$, the steady-state error $\lim_{k \rightarrow \infty} \|\mathbf{e}_k\|$ of the DNNTVTL model (3.8) for solving the time-varying TLS (1.3) is limited in $\frac{2n}{\theta_{\max}-1} \max_{0 \leq i \leq k, 1 \leq m \leq n} |\sigma_i^m| + \mathcal{O}(\tau^2)$, where $\sigma \in \mathbb{R}^n$ and $\theta_{\max} = \max\{1 + \theta\Psi(k\tau), 1 + \theta\Psi((k-1)\tau), \dots, 1 + \theta\Psi(\tau)\}$.

Proof From the proof of Theorem 4.3, we have

$$(1 + \theta\Psi(k\tau))\mathbf{e}_k - \mathbf{e}_{k-1} + \sigma_{k-1} = \mathbf{0}. \quad (4.18)$$

Consider the m -th subsystem, its display representation is

$$(1 + \theta\Psi(k\tau))\mathbf{e}_k^m - \mathbf{e}_{k-1}^m + \sigma_{k-1}^m = 0. \quad (4.19)$$

Thus,

$$\mathbf{e}_k^m = \frac{1}{1 + \theta\Psi(k\tau)}(\mathbf{e}_{k-1}^m - \sigma_{k-1}^m). \quad (4.20)$$

Consequently,

$$\begin{aligned} \mathbf{e}_k^m &= \frac{1}{1 + \theta\Psi(k\tau)}(\mathbf{e}_{k-1}^m - \sigma_{k-1}^m) \\ &= \frac{1}{1 + \theta\Psi(k\tau)} \left(\frac{1}{1 + \theta\Psi((k-1)\tau)}(\mathbf{e}_{k-2}^m - \sigma_{k-2}^m) - \sigma_{k-1}^m \right) \\ &\vdots \\ &= \frac{1}{\prod_{j=1}^k (1 + \theta\Psi(j\tau))} \mathbf{e}_0^m + \sum_{i=0}^{k-1} \frac{1}{\prod_{j=1}^{i+1} (1 + \theta\Psi(j\tau))} \sigma_i^m. \end{aligned}$$

Based on $\lim_{k \rightarrow \infty} \frac{1}{\prod_{j=1}^k (1 + \theta\Psi(j\tau))} = 0$, it follows from $\theta\Psi(j\tau) > 0$ for any $j > 0$ and $\tau > 0$ that

$$\lim_{k \rightarrow \infty} \mathbf{e}_k^m = \sum_{i=0}^{k-1} \frac{1}{\prod_{j=1}^{i+1} (1 + \theta\Psi(j\tau))} \sigma_i^m,$$

and

$$\begin{aligned} \lim_{k \rightarrow \infty} \|\mathbf{e}_k^m\| &= \left\| \sum_{i=0}^{k-1} \frac{1}{\prod_{j=1}^{i+1} (1 + \theta\Psi(j\tau))} \sigma_i^m \right\| \\ &\leq \sum_{i=0}^{k-1} \left\| \frac{1}{\prod_{j=1}^{i+1} (1 + \theta\Psi(j\tau))} \right\| |\sigma_i^m| \\ &\leq \max_{1 \leq i \leq k} |\sigma_i^m| \sum_{i=0}^{k-1} \left(\frac{1}{\theta_{\max}} \right)^{i+1} \\ &= \max_{0 \leq i \leq k} |\sigma_i^m| \left(\frac{1 - \frac{1}{\theta_{\max}}}{\frac{1}{\theta_{\max}} - 1} \right) \\ &\leq \left(\frac{1}{\theta_{\max} - 1} \right) \max_{0 \leq i \leq k} |\sigma_i^m| \\ &\leq \frac{2}{\theta_{\max} - 1} \max_{0 \leq i \leq k} |\sigma_i^m|. \end{aligned}$$

Considering the correlation between $\mathbf{e}_k$ and $\mathbf{e}_k^m$, we obtain.

$$\lim_{k \rightarrow \infty} \|\mathbf{e}_k\| \leq \frac{2n}{\theta_{\max}-1} \max_{0 \leq i \leq k, 1 \leq m \leq n} |\sigma_i^m|. \quad (4.21)$$

By integrating (4.21) with the $\mathcal{O}(\tau^2)$ term, we arrived at:

$$\lim_{k \rightarrow \infty} \|\mathbf{e}_k\| \leq \frac{2n}{\theta_{\max}-1} \max_{0 \leq i \leq k, 1 \leq m \leq n} |\sigma_i^m| + \mathcal{O}(\tau^2). \quad (4.22)$$

□

5 Numerical examples

We will simulate the proposed DNNTVTLS model (3.8) and illustrate its effectiveness and reliability for solving the TV matrix problem (1.4) in this section.

In the numerical tests, ℓ_2 norm of the error vector as t evolves is chosen to measure the residuals,

$$\text{Res} = \frac{\|\mathbf{A}_k \mathbf{x}_k - \mathbf{b}_k\|}{\sqrt{1 + \mathbf{x}_k^\top \mathbf{x}_k}}. \quad (5.1)$$

Furthermore, to estimate the approximate solution using proposed DNNTVTLS model (3.8), MATLAB code is utilized on a portable computer featuring a 2.50GHz Intel Core i5-4200M processor and 8GB of RAM.

In all tests, we adopt a termination criterion where either $\text{Res} \leq 1e-12$ or the maximum number of iterations exceeds 1000.

Example 5.1 (Cichocki and Unbehauen 1994) Let us consider the following linear parameter estimation problem described by the set of linear equations

$$\begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 2 \\ 2 \\ 2 \end{pmatrix}.$$

Let $\mathbf{x}_0 = (0, 0, 0)^\top$ be the starting vector in the computing process, and set the parameters $\gamma = 10$ and $\tau = 0.01$. The state variable trajectories by the DNNTVTLS model (3.8) are shown in Fig. 1a. The solution error (5.1) is displayed in Fig. 1b. From Fig. 1a, we find the approximate solution is $\mathbf{x}^* = (0.8544, 0.2587)^\top$, which is in good agreement with the exact results obtained by using MATLAB.

Example 5.2 (Gao et al. 1994) Assume that an “unknown” two-variable memoryless static systems is given by

$$w_1^* u_1(t) + w_2^* u_2(t) = v(t),$$

where w_1^* and w_2^* are the parameters to be estimated. In order to find the estimate of the unknown parameters, the input signals to the adaptive filter and the desired output signal $d(t)$ are generated from the expressions given by

$$\begin{cases} x_1(t) = u_1(t) + n_1(t), \\ x_2(t) = u_2(t) + n_2(t), \\ d(t) = v(t) + n_0(t), \end{cases}$$

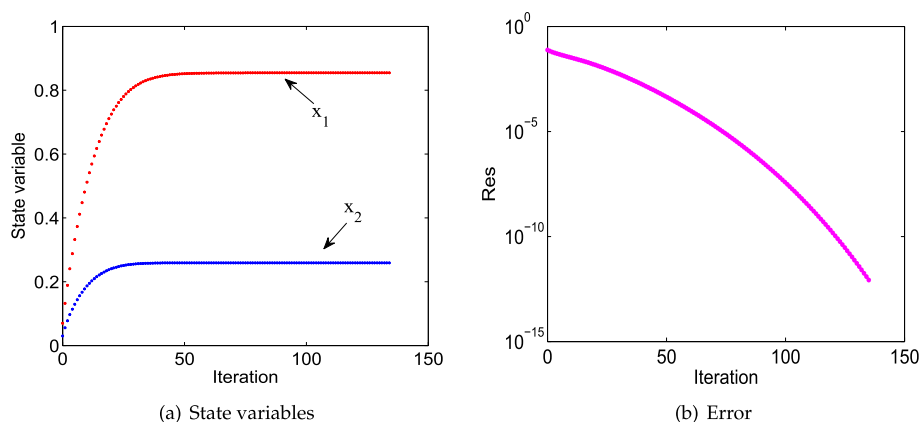


Fig. 1 Approximate solutions and error by the DNNTVTLS model for Example 5.1

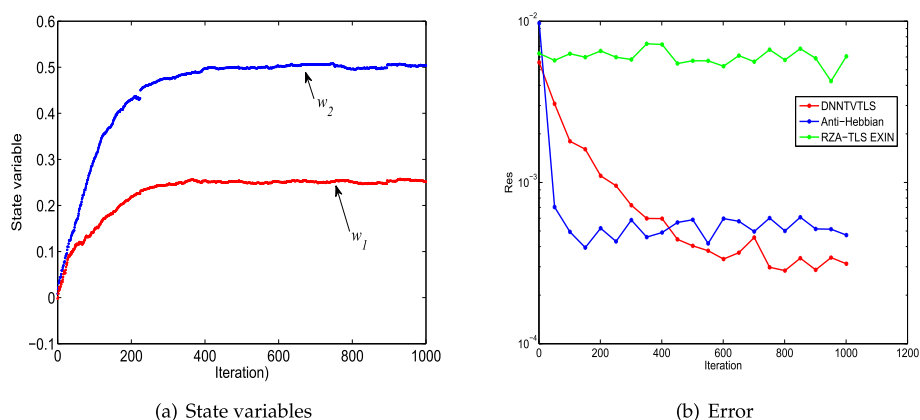


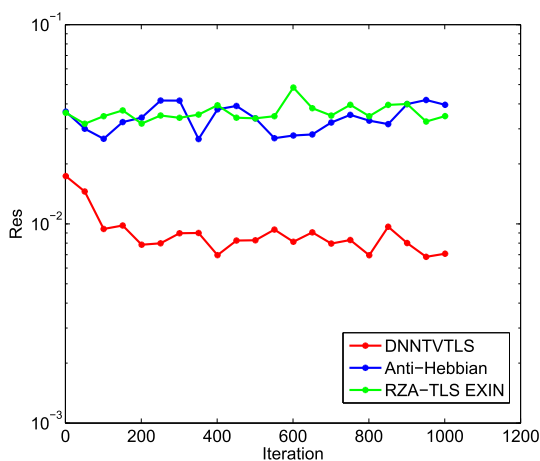
Fig. 2 The solution tracking curves and error variation of the DNNTVTLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for Example 5.2

where $u_1(t)$ and $u_2(t)$ are independent uniformly distributed random data over $[-0.5, 0.5]$, and $n_1(t)$, $n_2(t)$, and $n_0(t)$ are independent zero-mean white Gaussian noises, each with a variance of 0.01. Let the desired parameters be $w_1^* = 0.25$ and $w_2^* = 0.5$.

Take initial guess vector as $\mathbf{x}_0 = (0, 0, 0)^\top$, parameters $\gamma = 10$, $\tau = 0.0001$, $\alpha_1 = 1$, $\alpha_2 = 3$ and $\alpha_3 = 0.4$. Three adaptive algorithms are utilized for parameter estimation: the constrained anti-Hebbian algorithm presented in Gao et al. (1994), reweighted zero attractor total least squares by excitatory and inhibitory learning (RZA-TLS EXIN) proposed in Lim and Pang (2016) and the DNNTVTLS method.

The state variable trajectories by the DNNTVTLS model (3.8) are shown in Fig. 2a. Figure 2b depicts the averaged learning curves obtained from one hundred independent trials for each of the three adaptive approaches. The three algorithms yield comparable final outcomes. However, the DNNTVTLS method exhibits a smaller estimation error compared to both the constrained anti-Hebbian algorithm and the RZA-TLS EXIN method.

Fig. 3 The error variation of the DNNTV-TLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for Example 5.3



(a) Error

Table 1 Comparison results by using the DNNTV-TLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta \mathbf{A}_0)(t)$, $(\Delta \mathbf{b}_0)(t)$ and $\mathbf{n}_0(t)$

(m, n)	DNNTV-TLS			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.2971	6.2393e-11	18	0.6009	8.5493e-11	36	0.4533	6.6014e-11	28
(2000, 200)	0.6012	7.6296e-11	21	1.0282	9.3115e-11	36	0.7384	6.5180e-11	26
(2000, 500)	2.2793	8.5168e-11	20	4.009	6.9791e-11	38	2.9911	8.0839e-11	29

Example 5.3 (Gao et al. 1994) Let the parameter values of an “unknown” FIR system be given by

$$\mathbf{w}^* = (-0.3, -0.9, 0.8, -0.7, 0.6)^\top$$

and the output noise $n_0(t)$ be a zero-mean white Gaussian noise with a variance of 0.25. A random signal uniformly distributed over $[-0.5, 0.5]$ is used as the stimulating signal $\mathbf{u}(t)$. A white noise uniformly distributed over $[-k/2, k/2]$ serves as the additive input noise $n_i(t)$.

Let the initial guess vector be as $\mathbf{x}_0 = (0, 0, 0, 0, 0)^\top$, parameters $k = 0.5$, $\gamma = 4$, $\tau = 0.001$, $\alpha_1 = 1$, $\alpha_2 = 2.5$ and $\alpha_3 = 1$. Three adaptive algorithms are employed for parameter estimation: the constrained anti-Hebbian algorithm introduced in Gao et al. (1994), the RZA-TLS EXIN algorithm proposed in Lim and Pang (2016), and the DNNTV-TLS method. Figure 3 illustrates the averaged learning curves from one hundred independent trials for each of these three adaptive approaches. While all three algorithms achieve comparable final results, the DNNTV-TLS method demonstrates a lower estimation error in comparison to both the constrained anti-Hebbian algorithm and the RZA-TLS EXIN method.

Example 5.4 Let $\mathcal{A}(t) \in \mathbb{R}^{m \times n}$ be time-varying matrices flow with its entries

$$a_{ij}(t) = t \sin(i + j)$$

and the right-hand vector is $\mathbf{b}(t) = (1, 1, \dots, 1)^\top \in \mathbb{R}^m$.

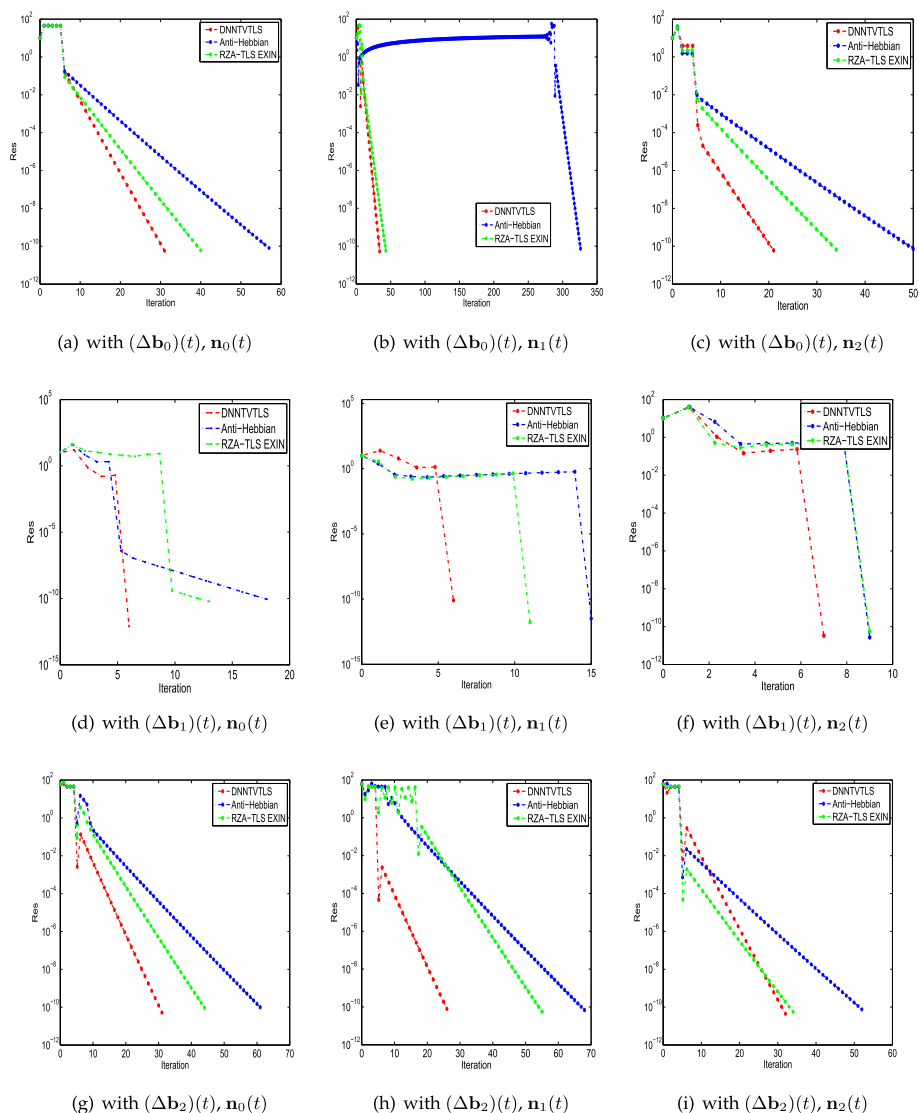


Fig. 4 Comparison results of the DNNTVTLs, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for $(\Delta A_0)(t) = 0.5 \times \text{ones}(m, n)$ in Example 5.4

Table 2 Comparison results by using the DNNTVTLs, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta A_0)(t)$, $(\Delta b_0)(t)$ and $n_1(t)$

(m, n)	DNNTVTLs			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.3380	5.8188e−11	22	0.6655	7.4027e−11	42	0.4433	9.7490e−11	29
(2000, 200)	0.5600	8.8434e−11	20	1.1349	8.0617e−11	42	0.6748	7.6114e−11	25
(2000, 500)	2.1229	5.0867e−11	21	4.2928	9.2906e−11	43	2.9549	7.7920e−11	29

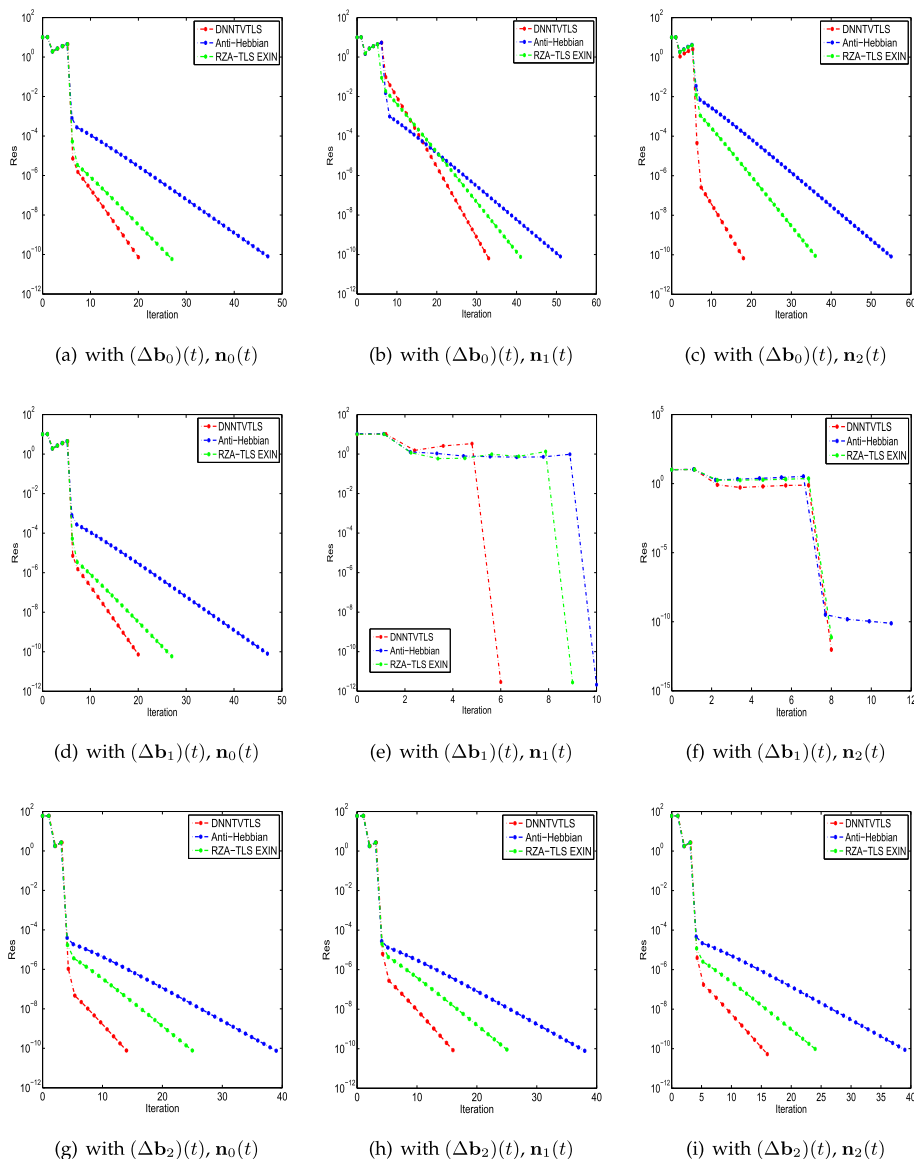


Fig. 5 Comparison results of the DNNTV-TLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for $(\Delta \mathbf{A}_1)(t) = 10t \times \text{ones}(m, n)$ in Example 5.4

Let the perturbation matrices be $(\Delta \mathbf{A}_0)(t) = 0.5 \times \text{ones}(m, n)$ and $(\Delta \mathbf{A}_1)(t) = 10t \times \text{ones}(m, n)$, perturbation vectors be $(\Delta \mathbf{b}_0)(t) = 20t \times \text{ones}(m, 1)$, $(\Delta \mathbf{b}_1)(t) = 0.25 \times \text{rand}(m, 1)$ and $(\Delta \mathbf{b}_2)(t) = 0.5 \times \text{ones}(m, 1)$, respectively. Noise vectors are $\mathbf{n}_0(t) = 10 \times \text{ones}(m, 1)$, $\mathbf{n}_1(t) = 10t \times \text{ones}(m, 1)$ and the random noise $\mathbf{n}_2(t) \in [5, 8]$, respectively.

To get the solution, we take initial vector as $\mathbf{x}(0) = (0, 0, \dots, 0)^\top$. For evaluating the accuracy of solution computed by models, we present the changes of the errors as time evolves for $m = 100$ and $n = 80$ of the coefficient matrix and right-hand vector.

Table 3 Comparison results by using the DNNTVTLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta \mathbf{A}_0)(t)$, $(\Delta \mathbf{b}_0)(t)$ and $\mathbf{n}_2(t)$

(m, n)	DNNTVTLS			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.4749	7.5486e−11	19	1.065	9.7652e−11	41	0.7822	9.6121e−11	27
(2000, 200)	0.9181	5.4057e−11	20	1.9855	7.0631e−11	45	1.1684	6.1928e−11	26
(2000, 500)	3.1873	6.5879e−11	20	6.7045	9.9017e−11	44	4.2501	6.0253e−11	27

The results computed by the DNNTVTLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN with the perturbation matrices $(\Delta \mathbf{A}_0)(t)$, $(\Delta \mathbf{A}_1)(t)$ and perturbation right-hand vectors $(\Delta \mathbf{b}_0)(t)$, $(\Delta \mathbf{b}_1)(t)$, $(\Delta \mathbf{b}_2)(t)$ and noise vectors $\mathbf{n}_0(t)$, $\mathbf{n}_1(t)$ and $\mathbf{n}_2(t)$ are presented in Figs. 4 and 5, respectively. Parameters are set as follows: $\gamma = 50$ together with $\tau = 0.001$ and $\alpha_1 = 1$, $\alpha_2 = 2.5$ and $\alpha_3 = 0.2$.

From Figs. 4 and 5, we can find that the DNNTVTLS model (3.8) has better convergence results compared with other methods.

Example 5.5 Let $\mathcal{A}(t) \in \mathbb{R}^{m \times n}$ be time-varying matrices flow with its entries

$$a_{ij}(t) = t(i + j)\sin(t)$$

and the right-hand vector is $\mathbf{b}(t) = \frac{\cos(t)}{m+n} \times \text{ones}(m, 1)$.

Let the perturbation matrices $(\Delta \mathbf{A}_0)(t)$, $(\Delta \mathbf{A}_1)(t)$ and perturbation right-hand vectors $(\Delta \mathbf{b}_0)(t)$, $(\Delta \mathbf{b}_1)(t)$, $(\Delta \mathbf{b}_2)(t)$ and noise vectors $\mathbf{n}_0(t)$, $\mathbf{n}_1(t)$ and $\mathbf{n}_2(t)$ be the same as Example 5.4. Parameters are set as follows: $\gamma = 50$ together with $\tau = 0.001$ and $\alpha_1 = 1$, $\alpha_2 = 2.5$ and $\alpha_3 = 0.2$. We compare the DNNTVTLS with constrained anti-Hebbian algorithm and RZA-TLS EXIN on CPU times, iteration steps and RES for different m and n . The comparison results are shown in Tables 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17 and 18.

From the above illustrative results computed by different models, the following conclusions can be drawn as follows,

- The proposed DNNTVTLS model (3.8) models can solve the TVTLS problem (1.4) efficiently.
- The proposed DNNTVTLS model (3.8) models demonstrates robust performance against both linear noise and random noise.
- The model DNNTVTLS model (3.8) converges to the total least squares solution and its convergence outperforms the constrained anti-Hebbian algorithm and RZA-TLS EXIN method under the same conditions.

6 Conclusion

In this paper, we introduce a neural network model specifically designed to address the time-varying total least squares (TVTLS) problem, termed the Neural Network for the Time-Varying Total Least Squares (NNTVTLS). We further derive the discretized form of this NNTVTLS model. Additionally, we devise an iterative procedure to obtain for the total least squares solution and conduct a thorough analysis of the convergence and robustness of the proposed discretized neural network for the time-varying total least squares (DNNTVTLS)

Table 4 Comparison results by using the DNNTV/TLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta \mathbf{A}_0)(t)$, $(\Delta \mathbf{b}_1)(t)$ and $\mathbf{n}_0(t)$

(m, n)	DNNTV/TLS			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.8739	4.6533e-11	38	1.3644	7.9022e-11	63	1.2448	7.1937e-11	54
(2000, 200)	1.9362	8.3657e-11	51	2.3240	7.3966e-11	63	2.3112	6.6388e-11	62
(2000, 500)	7.1055	7.6857e-11	51	10.2694	7.7215e-11	78	7.8867	9.4852e-11	59

Table 5 Comparison results by using the DNNTVTLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta \mathbf{A}_0)(t)$, $(\Delta \mathbf{b}_1)(t)$ and $\mathbf{n}_1(t)$

(m, n)	DNNTVTLS			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.2196	5.9068e-11	14	0.4693	9.5267e-11	29	0.3355	7.6059e-11	21
(2000, 200)	0.4825	5.1714e-11	17	0.6633	7.2961e-11	25	0.5404	6.6610e-11	20
(2000, 500)	1.4622	8.4498e-11	14	2.8253	9.7925e-11	27	1.7584	9.5448e-11	17

Table 6 Comparison results by using the DNNTVTLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta \mathbf{A}_0)(t)$, $(\Delta \mathbf{b}_1)(t)$ and $\mathbf{n}_2(t)$

(m, n)	DNNTVTLS			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.6549	9.8514e-11	41	1.0720	7.9119e-11	68	0.8337	9.9038e-11	53
(2000, 200)	1.4128	9.9829e-11	52	2.2964	8.3785e-11	84	1.6468	8.8581e-11	60
(2000, 500)	4.2654	6.5550e-11	43	7.7093	7.2938e-11	77	6.1923	9.4185e-11	61

Table 7 Comparison results by using the DNNTVTLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta \mathbf{A}_0)(t)$, $(\Delta \mathbf{b}_2)(t)$ and $\mathbf{n}_0(t)$

(m, n)	DNNTVTLS			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.3734	7.4625e-11	22	0.7826	7.7469e-11	48	0.4991	9.4701e-11	32
(2000, 200)	0.6055	9.9829e-11	22	21.2319	9.5102e-11	45	0.7942	7.5884e-11	29
(2000, 500)	2.4088	5.4536e-11	24	4.0347	7.5596e-11	41	2.6692	9.9908e-11	26

Table 8 Comparison results by using the DNNTVTLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta \mathbf{A}_0)(t)$, $(\Delta \mathbf{b}_2)(t)$ and $\mathbf{n}_1(t)$

(m, n)	DNNTVTLS			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.4877	4.8244e-11	31	0.9874	9.6779e-11	62	0.7393	7.3624e-11	47
(2000, 200)	0.5808	5.7767e-11	22	1.8760	7.7518e-11	68	1.0098	8.2043e-11	37
(2000, 500)	2.4232	8.4510e-11	24	7.3251	9.9361e-11	71	4.0149	8.3849e-11	41

Table 9 Comparison results by using the DNNTVTLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta \mathbf{A}_0)(t)$, $(\Delta \mathbf{b}_2)(t)$ and $\mathbf{n}_2(t)$

(m, n)	DNNTVTLS			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.3519	5.8031e-11	22	0.6708	8.3254e-11	40	0.4829	6.6322e-11	30
(2000, 200)	0.6078	8.6922e-11	22	1.2355	7.2663e-11	44	0.7722	6.6670e-11	28
(2000, 500)	2.0848	7.8837e-11	21	4.3816	7.2333e-11	45	3.0054	5.8029e-11	30

Table 10 Comparison results by using the DNNTVTLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta \mathbf{A}_1)(t)$, $(\Delta \mathbf{b}_0)(t)$ and $\mathbf{n}_0(t)$

(m, n)	DNNTVTLS			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.3935	8.6576e-11	26	0.7823	9.9722e-11	51	0.5493	6.4734e-11	36
(2000, 200)	0.7109	4.5047e-11	26	1.3800	8.7813e-11	50	0.9493	6.1983e-11	35
(2000, 500)	2.2132	5.0870e-11	23	4.7485	6.9878e-11	50	3.2779	5.7844e-11	34

Table 11 Comparison results by using the DNNTVTLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta \mathbf{A}_1)(t)$, $(\Delta \mathbf{b}_0)(t)$ and $\mathbf{n}_1(t)$

(m, n)	DNNTVTLS			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.6123	4.1211e-11	25	1.1439	9.7668e-11	44	0.8647	6.8624e-11	35
(2000, 200)	1.0878	7.5392e-11	25	1.8155	9.1815e-11	41	1.5684	7.4255e-11	35
(2000, 500)	3.8386	5.1909e-11	26	6.3749	4.2969e-11	38	4.7115	9.9083e-11	32

Table 12 Comparison results by using the DNNTVTLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta \mathbf{A}_1)(t)$, $(\Delta \mathbf{b}_0)(t)$ and $\mathbf{n}_2(t)$

(m, n)	DNNTVTLS			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.5740	8.4645e-11	23	1.2212	9.1293e-11	49	0.7682	5.8564e-11	32
(2000, 200)	1.0106	5.8364e-11	24	1.9463	6.9455e-11	47	1.3374	7.3681e-11	32
(2000, 500)	2.2141	5.3289e-11	24	4.3851	7.0378e-11	48	2.8869	9.0471e-11	32

Table 13 Comparison results by using the DNNTVTLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta \mathbf{A}_1)(t)$, $(\Delta \mathbf{b}_1)(t)$ and $\mathbf{n}_0(t)$

(m, n)	DNNTVTLS			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.2490	9.7048e-11	12	0.3760	7.9645e-11	18	0.2375	9.9328e-11	11
(2000, 200)	0.4704	9.9246e-11	12	0.9552	8.0277e-11	25	0.4762	9.2912e-11	13
(2000, 500)	1.7687	4.7597e-11	20	1.3939	9.8364e-11	16	1.8459	6.5973e-11	21

Table 14 Comparison results by using the DNNTVTLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta \mathbf{A}_1)(t)$, $(\Delta \mathbf{b}_1)(t)$ and $\mathbf{n}_1(t)$

(m, n)	DNNTVTLS			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.1749	8.6762e-11	7	0.4455	7.4529e-11	21	0.1894	9.0454e-11	9
(2000, 200)	0.3907	4.9563e-11	11	0.8073	8.4239e-11	22	0.6452	6.3845e-11	18
(2000, 500)	1.2378	4.7686e-13	10	3.4308	7.8367e-11	27	2.3811	6.0672e-11	19

Table 15 Comparison results by using the DNNTVTLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta \mathbf{A}_1)(t)$, $(\Delta \mathbf{b}_1)(t)$ and $\mathbf{n}_2(t)$

(m, n)	DNNTVTLS			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.2015	8.2030e-11	7	0.5225	9.7931e-11	24	0.2338	8.3580e-11	11
(2000, 200)	0.2765	5.2046e-11	8	0.7620	7.3502e-11	21	0.4139	6.9739e-11	11
(2000, 500)	1.1394	6.6454e-11	9	1.8249	8.3601e-11	15	1.3393	8.0367e-11	11

Table 16 Comparison results by using the DNNTVTLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta \mathbf{A}_1)(t)$, $(\Delta \mathbf{b}_2)(t)$ and $\mathbf{n}_0(t)$

(m, n)	DNNTVTLS			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.6155	6.1550e-11	30	1.0740	9.6720e-11	51	0.7598	7.9863e-11	37
(2000, 200)	0.8155	8.7268e-11	24	1.5397	8.0696e-11	44	1.0963	8.7488e-11	36
(2000, 500)	2.5000	9.9337e-11	21	5.8138	9.1457e-11	48	3.5721	8.2848e-11	30

Table 17 Comparison results by using the DNNTVTLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta \mathbf{A}_1)(t)$, $(\Delta \mathbf{b}_2)(t)$ and $\mathbf{n}_1(t)$

(m, n)	DNNTVTLS			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.5070	6.9760e-11	22	1.3312	6.9401e-11	57	0.7423	9.0539e-11	33
(2000, 200)	0.9360	6.8430e-11	26	1.7745	9.4204e-11	48	1.2906	5.8975e-11	35
(2000, 500)	2.8773	8.3975e-11	23	6.0428	7.8838e-11	48	4.3730	5.7627e-11	34

Table 18 Comparison results by using the DNNTVTLS, constrained anti-Hebbian algorithm and the RZA-TLS EXIN for different m and n with $(\Delta \mathbf{A}_1)(t)$, $(\Delta \mathbf{b}_2)(t)$ and $\mathbf{n}_2(t)$

(m, n)	DNNTVTLS			Anti-Hebbian			RZA-TLS EXIN		
	CPU	RES	IT	CPU	RES	IT	CPU	RES	IT
(1000, 200)	0.4654	7.1228e-11	24	1.0295	7.0745e-11	66	0.6511	8.2084e-11	42
(2000, 200)	0.6813	7.6414e-11	25	1.2411	7.1762e-11	45	1.0225	6.1889e-11	33
(2000, 500)	2.1438	7.3189e-11	22	4.7769	9.7892e-11	44	3.6018	6.2050e-11	32

model. Simulations demonstrate the superiority of the DNNTVTLS model over the RZA-TLS EXIN and anti-Hebbian algorithms with various perturbation matrices and noisy vectors.

Acknowledgements The authors would like to thank the handling editor and two anonymous reviewers for their detailed and valuable comments.

Data availability Not applicable.

Declarations

Conflict of interest The authors have no conflict of interest to declare.

References

- Bunse-Gerstner A, Byers R, Mehrmann V, Nichols NK (1991) Numerical computation of an analytic singular value decomposition of a matrix valued function. *Numer Math* 60:1–39
- Cichocki A, Unbehauen R (1994) Simplified neural networks for solving linear least squares and total least squares problems in real time. *IEEE Trans Neural Netw* 5:910–923
- Cirrincone G, Cirrincone M (1999) Linear system identification using the TLS EXIN neuron. *Neurocomputing* 28:53–74
- Dai J, Tan P, Xiao L, Jia L, He Y, Luo J (2023) A fuzzy adaptive zeroing neural network model with event-triggered control for time-varying matrix inversion. *IEEE Trans Fuzzy Syst* 31:3974–3983
- Fierro RD, Golub GH, Hansen PC, O’Leary DP (1997) Regularization by truncated total least squares. *SIAM J Sci Comput* 18:1223–1241
- Gao K, Ahmad MO, Swamy MNS (1994) A constrained anti-Hebbian learning algorithm for total least-squares estimation with applications to adaptive fir and iir filtering. *IEEE Trans Circuits Syst II Analog Digit Signal Process* 41:718–729
- Golub GH, van Loan CF (1980) An analysis of the total least squares problem. *SIAM J Numer Anal* 17:883–893
- Golub G, Van Loan C (2014) *Matrix computations*, 4th edn. The Johns Hopkins University Press, Baltimore
- Golub GH, Hansen PC, O’Leary DP (1999) Tikhonov regularization and total least squares. *SIAM J Matrix Anal Appl* 21:185–194
- Guo D, Yi C, Zhang Y (2011) Zhang neural network versus gradient-based neural network for time-varying linear matrix equation solving. *Neurocomputing* 74:3708–3712
- Han F, Wei Y, Xie P (2024) Regularized and structured tensor total least squares methods with applications. *J Optim Theory Appl* 202:1101–1136
- Huffel SV, Vandewalle J (1991) Total least squares problem—computational aspects and analysis, vol. 9 of *frontiers in applied mathematics*. SIAM
- Jin L, Zhang Y (2014) Discrete-time Zhang neural network of $o(\tau^3)$ pattern for time-varying matrix pseudoinversion with application to manipulator motion generation. *Neurocomputing* 142:165–173
- Jin L, Zhang Y, Li S (2016) Integration-enhanced Zhang neural network for real-time-varying matrix inversion in the presence of various kinds of noises. *IEEE Trans Neural Netw Learn Syst* 27:2615–2627
- Katsikis VN, Stanimirović PS, Mourtas SD, Xiao L, Karabašević D, Stanujkić D (2022) Zeroing neural network with fuzzy parameter for computing pseudoinverse of arbitrary matrix. *IEEE Trans Fuzzy Syst* 30:3426–3435
- Kong X, Feng D (2024) *Efficient online learning algorithms for total least square problems, engineering applications of computational methods*, vol 21. Springer, Science Press, Berlin
- Lei Y, Dai Z, Liao B, Xia G, He Y (2022) Double features zeroing neural network model for solving the pseudoinverse of a complex-valued time-varying matrix. *Mathematics* 10:2122
- Li S, Li Y (2014) Nonlinearly activated neural network for solving time-varying complex Sylvester equation. *IEEE Trans Cybern* 44:1397–1407
- Liao B, Xiang Q, Li S (2019) Bounded z-type neurodynamics with limited-time convergence and noise tolerance for calculating time-dependent Lyapunov equation. *Neurocomputing* 325:234–241
- Liao B, Wang Y, Li W, Peng C, Xiang Q (2021) Prescribed-time convergent and noise-tolerant z-type neural dynamics for calculating time-dependent quadratic programming. *Neural Comput Appl* 33:5327–5337
- Lim JS, Pang H (2016) Reweighted l_1 regularized TLS linear neuron for the sparse system identification. *Neurocomputing* 173:1972–1975
- Liu Q, Jia Z, Wei Y (2022) Multidimensional total least squares problem with linear equality constraints. *SIAM J Matrix Anal Appl* 43:124–150
- Lu S, Pereverzev SV, Tautenhahn U (2009) Regularized total least squares: computational aspects and error bounds. *SIAM J Matrix Anal Appl* 31:918–941
- Luo FL, Unbehauen R (1999) *Applied neural networks for signal processing*. Cambridge University Press, Cambridge
- Mo C, Wang X, Wei Y (2020) Time-varying generalized tensor eigen analysis via Zhang neural networks. *Neurocomputing* 407:465–479
- Perez-Ilzarbe M (1998) Convergence analysis of a discrete-time recurrent neural network to perform quadratic real optimization with bound constraints. *IEEE Trans Neural Netw* 9:1344–1351

- Petković MD, Stanimirović PS, Katsikis VN (2018) Modified discrete iterations for computing the inverse and pseudoinverse of the time-varying matrix. *Neurocomputing* 289:155–165
- Qi Z, Ning Y, Xiao L, Wang Z, He Y (2025) Efficient predefined-time adaptive neural networks for computing time-varying tensor Moore–Penrose inverse. *IEEE Trans Neural Netw Learn Syst*. <https://doi.org/10.1109/TNNLS.2024.3354936>
- Qiao S, Wang X, Wei Y (2018) Two finite-time convergent Zhang neural network models for time-varying complex matrix Drazin inverse. *Linear Algebra Appl* 542:101–117
- Qiao S, Wei Y, Zhang X (2020) Computing time-varying ML-weighted pseudoinverse by the Zhang neural networks. *Numer Funct Anal Optim* 41:1672–1693
- Qiu B, Zhang Y (2019) Two new discrete-time neurodynamic algorithms applied to online future matrix inversion with nonsingular or sometimes-singular coefficient. *IEEE Trans Cybern* 49:2032–2045
- Shen Y, Miao P, Huang Y, Shen Y (2015) Finite-time stability and its application for solving time-varying Sylvester equation by recurrent neural network. *Neural Process Lett* 42:763–784
- Shi Y, Zhao W-G, Li S, Li B, Sun X (2022) Direct derivation scheme of DT-RNN algorithm for discrete time-variant matrix pseudo-inversion with application to robotic manipulator. *Appl Soft Comput* 133:109861
- Shi Y, Chong W, Zhao W-G, Li S, Li B, Sun X (2023) A new recurrent neural network based on direct discretization method for solving discrete time-variant matrix inversion with application. *Inf Sci* 652:119729
- Wang X, Wei Y, Stanimirović PS (2016) Complex neural network models for time-varying Drazin inverse. *Neural Comput* 28:2790–2824
- Wang X, Mo C, Qiao S, Wei Y (2022) Predefined-time convergent neural networks for solving the time-varying nonsingular multi-linear tensor equations. *Neurocomputing* 472:68–84
- Wei Y, Stanimirović P, Petković M (2018) Numerical and symbolic computations of generalized inverses. World Scientific, Hackensack
- Xiao L (2015) A finite-time convergent neural dynamics for online solution of time-varying linear complex matrix equation. *Neurocomputing* 167:254–259
- Xiao L, Zhang Y (2011) Zhang neural network versus gradient neural network for solving time-varying linear inequalities. *IEEE Trans Neural Netw* 22:1676–84
- Xiao L, Zhang Y (2012) Two new types of Zhang neural networks solving systems of time-varying nonlinear inequalities. *IEEE Trans Circuits Syst I*(59):2363–2373
- Xiao L, Zhang Y (2013) From different Zhang functions to various ZNN models accelerated to finite-time convergence for time-varying linear matrix equation. *Neural Process Lett* 39:309–326
- Xiao L, Zhang Y, Zuo Q, Dai J, Li J, Tang W (2020) A noise-tolerant zeroing neural network for time-dependent complex matrix inversion under various kinds of noises. *IEEE Trans Ind Inf* 16:3757–3766
- Xiao L, Li L, Huang W, Li X, Jia L (2024) A new predefined time zeroing neural network with drop conservatism for matrix flows inversion and its application. *IEEE Trans Cybern* 54:752–761
- Xie P, Xiang H, Wei Y (2017) A contribution to perturbation analysis for total least squares problems. *Numer Algorithms* 75:381–395
- Xie P, Xiang H, Wei Y (2018) Randomized algorithms for total least squares problems. *Numer Linear Algebra Appl* 26:e2219
- Zhang Y, Ge SS (2005) Design and analysis of a general recurrent neural network model for time-varying matrix inversion. *IEEE Trans Neural Netw* 16:1477–1490
- Zhang Y, Yi C (2011) Zhang neural networks and neural-dynamic method. Nova Science Publishers, Inc., Hauppauge
- Zhang Y, Jiang D, Wang J (2002) A recurrent neural network for solving Sylvester equation with time-varying coefficients. *IEEE Trans Neural Netw* 13:1053–63
- Zhang Y, Ma W, Cai B (2009) From Zhang neural network to Newton iteration for matrix inversion. *IEEE Trans Circuits Syst I*(56):1405–1415
- Zhang Y, Ling Y, Yang M, Yang S, Zhang Z (2021) Inverse-free discrete ZNN models solving for future matrix pseudoinverse via combination of extrapolation and ZeaD formulas. *IEEE Trans Neural Netw Learn Syst* 32:2663–2675
- Zheng B, Meng L, Wei Y (2017) Condition numbers of the multidimensional total least squares problem. *SIAM J Matrix Anal Appl* 38:924–948

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.